Model Context Protocol

MCP & MCP Security

A simple, practical explanation of how AI agents connect to tools and enterprise systems—and how to secure that connection.

TOOLS

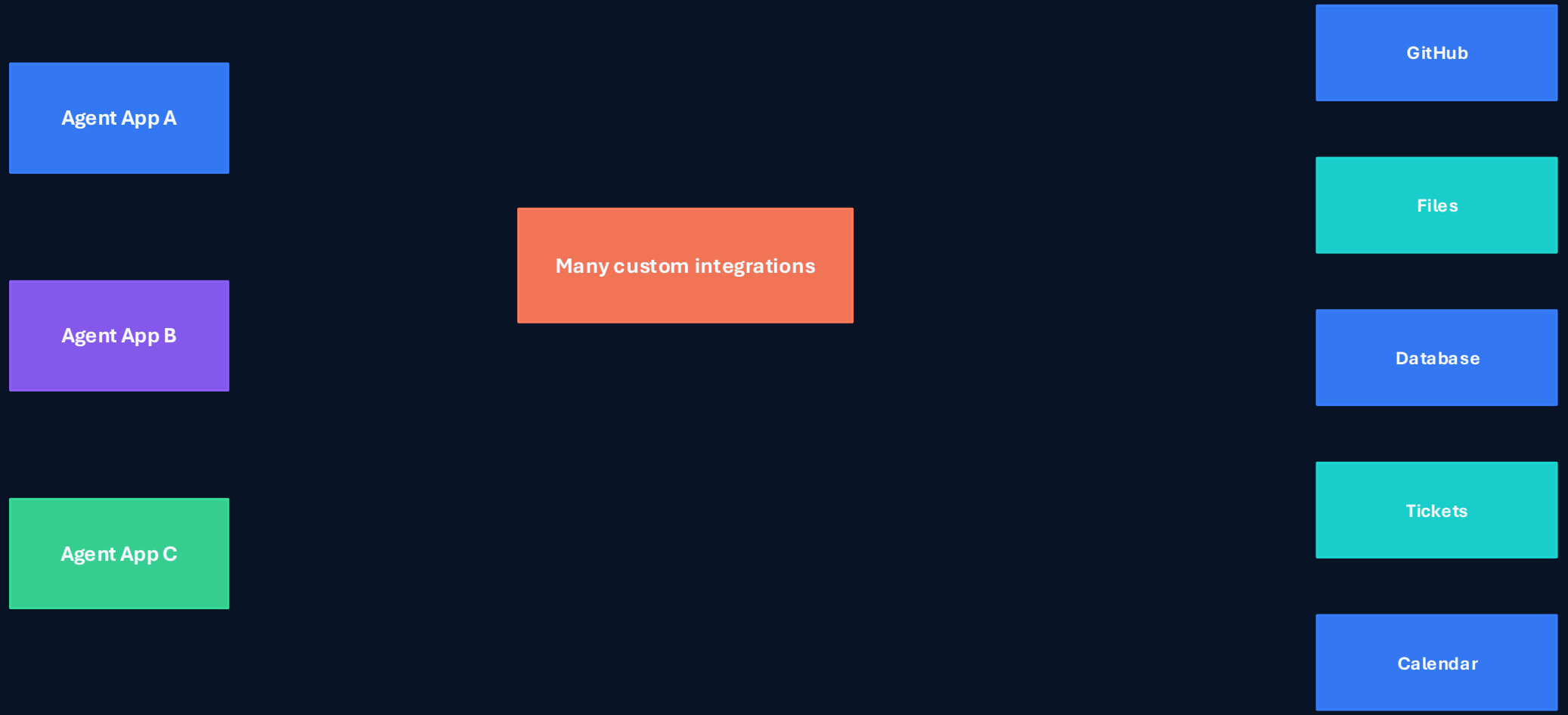
RESOURCES

PROMPTS

SECURITY

Why MCP is needed

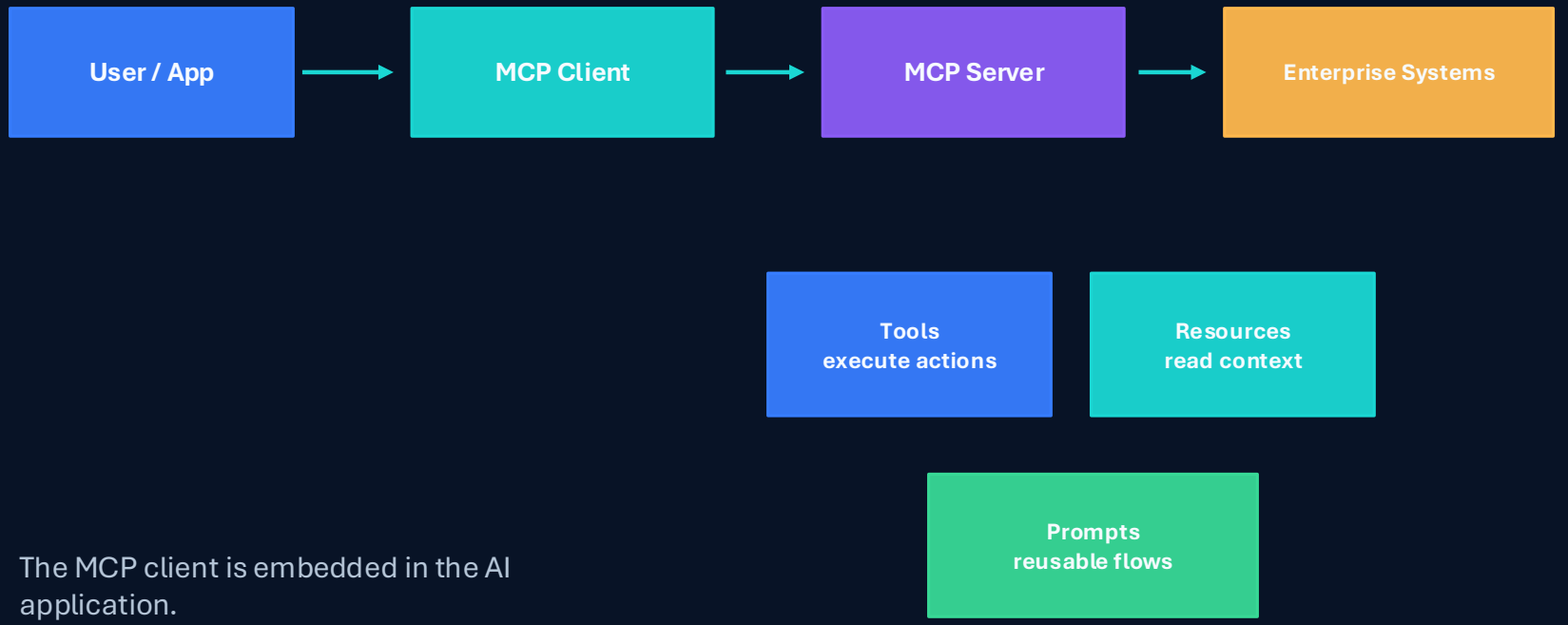
The integration problem appears when every agent needs custom connectors.



Without MCP, each app needs custom API code, auth, schemas, retries, and security checks.
MCP creates a common way for AI systems to discover and use external capabilities.

MCP core architecture

Client, server, tools, resources, prompts, and transport.



The MCP client is embedded in the AI application.
The MCP server exposes capabilities in a standard format.
The transport can be stdio or HTTP depending on deployment needs.

Simple GitHub MCP configuration

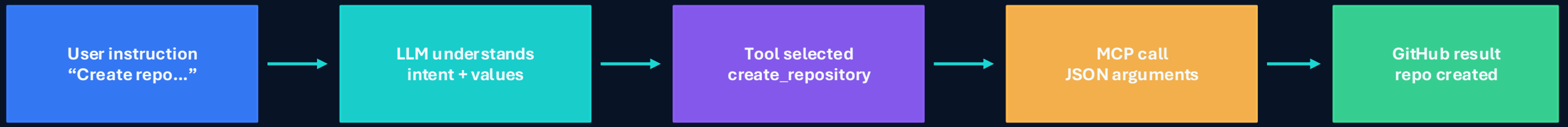
A clean setup uses one GitHub server configuration, one server launcher, and one client.


```
{ "mcpServers": { "github": { "command": "npx", "args": ["-y", "@modelcontextprotocol/server-github"], "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "..." } } } }
```

The configuration tells the client how to start the GitHub MCP server.
The server exposes GitHub tools; the client discovers them and calls them.

Natural-language GitHub workflow

The user describes the task; the LLM selects the MCP tool and arguments.

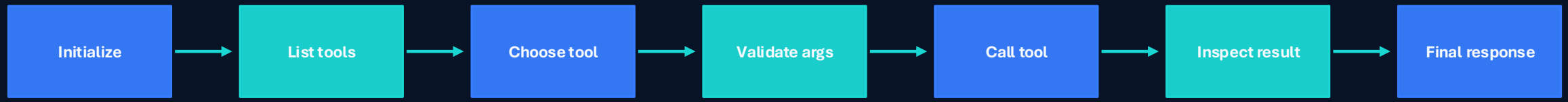


Example commands

Create a private GitHub repository named mcp-day3-demo.
Create README.md with a short project description.
Create an issue titled “Add authentication”.
Search repositories related to MCP Python examples.

MCP tool lifecycle

Every tool call should be discoverable, structured, observable, and validated.



Tool discovery prevents hard-coded assumptions about server capabilities.
Tool schemas tell the model which arguments are required.
Validation prevents empty queries, wrong paths, unsafe actions, and unnecessary API failures.

```
Bad tool call: search_repositories({ query: "" })

Safe client: block and ask model to correct arguments
```

MCP security risk map

The MCP boundary is powerful because it connects AI decisions to real tools.

- Untrusted server
- Overpowered token
- Tool misuse
- Prompt injection
- Result poisoning
- Unsafe writes
- Secret leakage
- No audit trail

MCP security is not only about the protocol.
It is about deciding which server is trusted, which tools are allowed, which parameters are valid, and which actions require approval.

Secure MCP client controls

The client is the policy enforcement point before tools execute.

Input check

Block obvious bypass prompts

Tool allow-list

Expose only approved tools

Parameter validation

Reject empty query / bad paths

Approval gate

Confirm writes

Tool-call limit

Stop loops

Audit log

Record decisions

Security rule: never let the LLM directly execute tools without application-side policy checks.

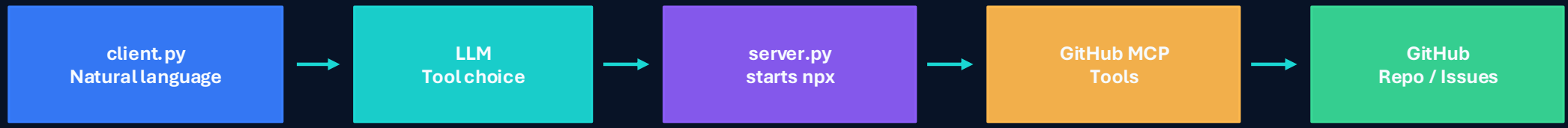
Tool permissions & approval gates

Read operations can be automatic; write operations need stronger control.

A GitHub agent that can create repositories and files can change real systems.
Permission scoping, human approval, and audit logs are mandatory for write actions.

Generic GitHub MCP example

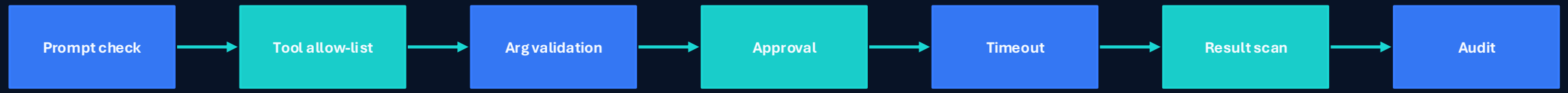
Start with the simple client-server pattern before adding controls.



Lab flow: create repository → create README → create issue → list tools → inspect MCP results

GitHub MCP with security

The secure version uses the same architecture plus policy checks.



Decision outcomes

ALLOW

BLOCK

REVIEW

APPROVAL_REQUIRED

The secure client protects against wrong tool choice, empty search queries, unsafe write actions, indirect prompt injection in GitHub content, and missing audit evidence.

Key takeaways & lab checklist

MCP gives agents capabilities; security decides how those capabilities are used.

1. Start with generic MCP
2. Add validation
3. Gate write actions
4. Inspect tool results
5. Audit everything

Checklist:

- ✓ GitHub token in .env, not in code
- ✓ npm/npx available
- ✓ list_tools() works
- ✓ natural language triggers the right tool
- ✓ empty/unsafe arguments are blocked
- ✓ write actions require approval
- ✓ logs capture decisions and results